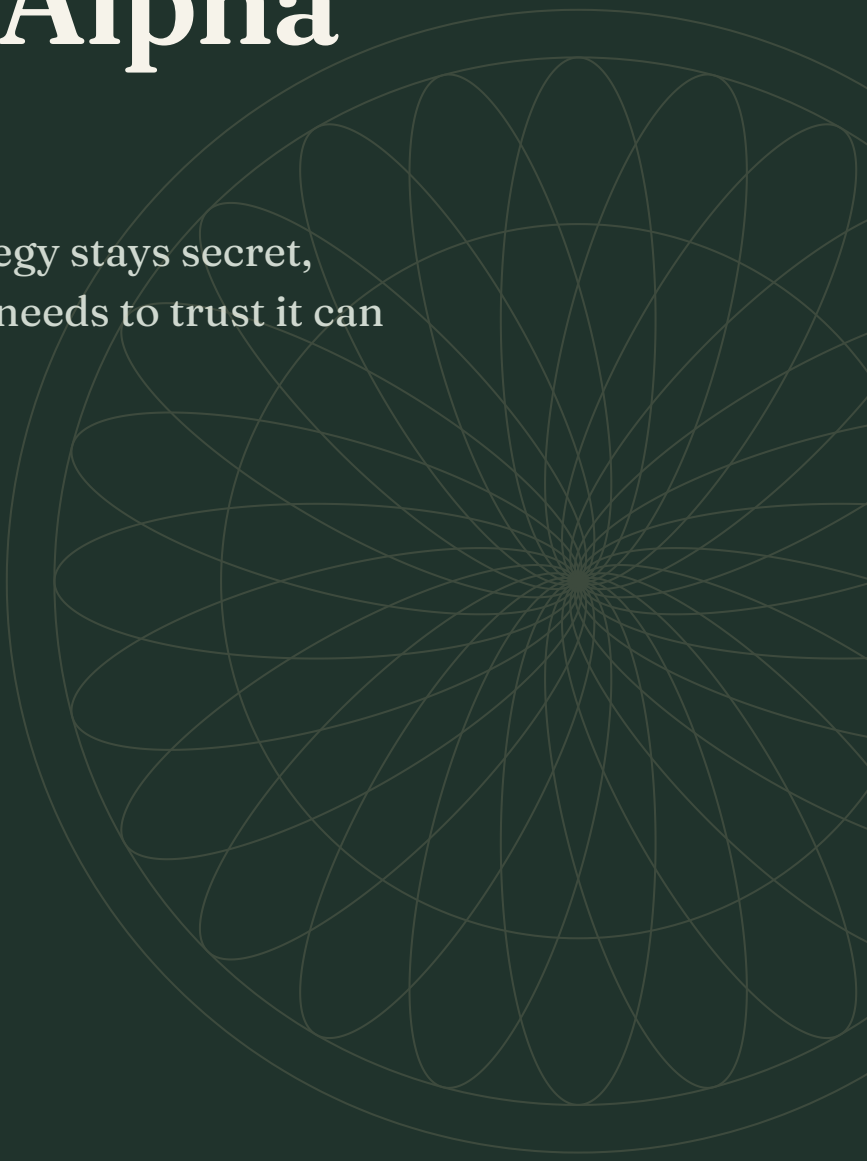
PRODUCT DOCUMENT · NEAR HACKATHON 2026

Verifiable Alpha Vaults

AI trading vaults whose strategy stays secret, while everything an investor needs to trust it can be checked by anyone.



Which code ran

Intel TDX attestation of the exact strategy build

No back-filled trades

Every decision hashed on NEAR before it executes

Risk limits held

Contract guardrails plus Bulletproofs, checked in your browser

Real returns

On-chain share accounting; trades signed by the vault itself

13 November 2026 · v5 · live on NEAR testnet

github.com/amudhan23/verifiable-alpha-vaults

CONTENTS

01 Summary

02 The problem

03 Who it is for: user stories

04 The solution

05 Architecture

06 One decision, end to end

07 How each part works, and what it solves

08 Trust model: who can do what

09 Money guarantees and security work

10 Live evidence

11 Limitations and roadmap

12 Glossary

01 · SUMMARY

Don't trust the fund manager. Check.

Verifiable Alpha Vaults lets anyone deposit into an AI-run trading vault without having to trust the person who runs it, and without the strategy being exposed to copycats.

A strategy is split into two parts. The **code** is public: anyone can read the logic and check its fingerprint. The **parameters**, the numbers that make it profitable, stay sealed inside Intel TDX secure hardware on NEAR's OutLayer network. Every decision the strategy makes is hashed onto NEAR before it executes. The vault contract refuses anything outside its risk limits, and zero-knowledge range proofs (Bulletproofs) show that private risk metrics stayed within bounds. The vault signs its own trades on Uniswap through NEAR Chain Signatures and brings value back through NEAR Intents.

Every one of these claims can be checked by an investor in a browser, with no special tools and no trust in us.

4 seals	56 + 4	2 reviews
Code attested, decisions committed, limits proven, prices checked	Rust tests (incl. property-based fuzz of the money rules) and executor tests	Independent security reviews; every finding fixed or disclosed

Status. Live on NEAR testnet with two vaults. A fully attested cycle has run end to end: the vault asked the TEE for a decision on-chain, received it in its own callback, and signed a real swap on Ethereum Sepolia. The mainnet build (trading on Base) is ready to deploy.

AI fund managers ask for blind trust

Trading agents and on-chain funds are multiplying, and almost all of them ask investors for the same thing: trust me.

An investor looking at an AI trading vault today cannot answer four basic questions:

1. **Which code is actually running?** The manager says "our model", but can change it at any time, silently.
2. **Is the track record real?** Trades can be back-filled, losing runs hidden, and only the winners shown.
3. **Were the risk limits respected?** "Max 3x leverage" is a sentence on a website, not something anyone enforces.
4. **Are the returns real?** Reported performance is rarely tied to money that actually moved.

The manager's dilemma

Strategy creators face the opposite problem. The obvious way to earn trust is to publish the strategy. But a published strategy is copied within days and its edge disappears. So managers keep strategies secret, and investors are left with no way to verify anything. Both sides lose: good strategies can't prove themselves, and investors can't tell good from bad.

> *The edge has to stay private. The behaviour has to be public. Until now you could only have one of the two.*

What exists today

Approach	Strategy private?	Investor can verify behaviour?	Money safe from the manager?
Traditional or black-box fund	Yes	No: audited monthly at best	Legally, not technically
Open-source strategy vault	No: copied immediately	Yes	Often
Signal sellers and copy-trading	Until sold, then leaked	Past signals only, cherry-pickable	Not applicable
AI agent with spending limits	Yes	Limits only, not decisions	Within the limits
Verifiable Alpha Vaults	**Yes: parameters sealed in hardware**	**Yes: every decision, limit and price**	**Yes: contract-enforced**

03 · WHO IT IS FOR

User stories

Five people use the product. Each story ends with how the product satisfies it today.

Investor deposits NEAR into a vault	As an investor, I want to know that the strategy I'm paying for is the one actually running, so that a manager can't swap in something riskier after I deposit.	**Today:** each decision carries an Intel TDX attestation of the exact build. The vault approves one build by its fingerprint, and changing it takes a public 48-hour timelock, so I can leave first.
Investor	As an investor, I want proof the track record wasn't back-filled or cherry-picked, so that I can judge the strategy on everything it did.	**Today:** every decision is hashed on NEAR before it executes, and the swap's own calldata ends in that hash. The ledger is complete by construction.
Investor	As an investor, I want to withdraw my exact share at any time, and get out even if the operator disappears.	**Today:** withdrawals pay an exact pro-rata slice of the vault's NEAR and never depend on anyone's valuation. If no decision arrives for 24 hours, anyone can trigger a one-way emergency exit.
Strategy manager a quant or AI builder	As a strategy creator, I want to prove my strategy is good without revealing it, so that I can attract capital without being copied.	**Today:** my code is public but my parameters live only inside TDX, released only to my exact build. My verified record and fee rights live in a strategy NFT I own and can sell.
Strategy manager	As a strategy creator, I want to earn fees only on real performance, so that investors believe the incentives are fair.	**Today:** 20% performance fee, only on realized gains above a high-water mark, minted to whoever holds the strategy NFT.
Sceptic auditor, journalist, judge	As a sceptic, I want to verify every claim myself in minutes, without trusting the team.	**Today:** the vault page runs four checks in the browser, `node executor/verify.mjs` runs six from a terminal without keys, and `VERIFY.md` maps each claim to code, tests and live transactions.
Strategy buyer acquires a track record	As a buyer, I want to buy a strategy together with its verified history and future fees.	**Today:** the strategy is a NEP-171 NFT. Its holder earns the fees and proposes changes; any proposal made by a previous owner lapses on sale.

04 · THE SOLUTION

Public code, private numbers, provable behaviour

The protected asset is the strategy's parameters and its future decisions. Everything else is public, and checkable.

Part	Public or private	Why
Strategy code (WASM)	**Public, fingerprinted**	Investors can read the logic; the vault approves one exact build.
Parameters	**Sealed in TDX**	The numbers are the edge. They exist in readable form only inside the enclave.
Each decision	Hashed first, revealed at execution	Can't be back-filled; can't be front-run before it's committed.
Risk metrics	Proven, not revealed	Bulletproofs show leverage and drawdown stayed within limits without exposing them.
Money	**Fully on-chain**	Shares, NAV, fees and withdrawals are contract state anyone can read.

The four seals

1 · Which code ran

OutLayer runs the strategy in Intel TDX and publishes an attestation naming the exact build it executed. The vault only accepts the build it approved.

2 · Committed before executed

The enclave outputs `sha256(decision || salt)`, which the vault stores before anything trades. The reveal must match it byte for byte.

3 · Within limits

The contract refuses decisions over its guardrails before any signature exists. Bulletproofs prove private metrics stayed in range; the browser verifies them in WebAssembly.

4 · Real prices

Every relayed decision publishes its exact input; its hash must match the attestation, and the prices are checked against public market history.

What makes it different

- **The vault trades for itself.** The contract builds and MPC-signs its own Uniswap transactions. The executor can't choose amounts or recipients; it just broadcasts.
- **Honest about trust.** Two explicit modes say exactly what the relayer is still trusted for, and the stronger mode removes it.
- **A way back.** Cross-chain value returns as NEAR through NEAR Intents, with a public delay before the vault signs anything. And every claim is checkable by anyone, with no keys.

05 · ARCHITECTURE

The system at a glance

Seven parts, each with one job. The vault contract on NEAR is the centre: it holds the money and enforces every rule.

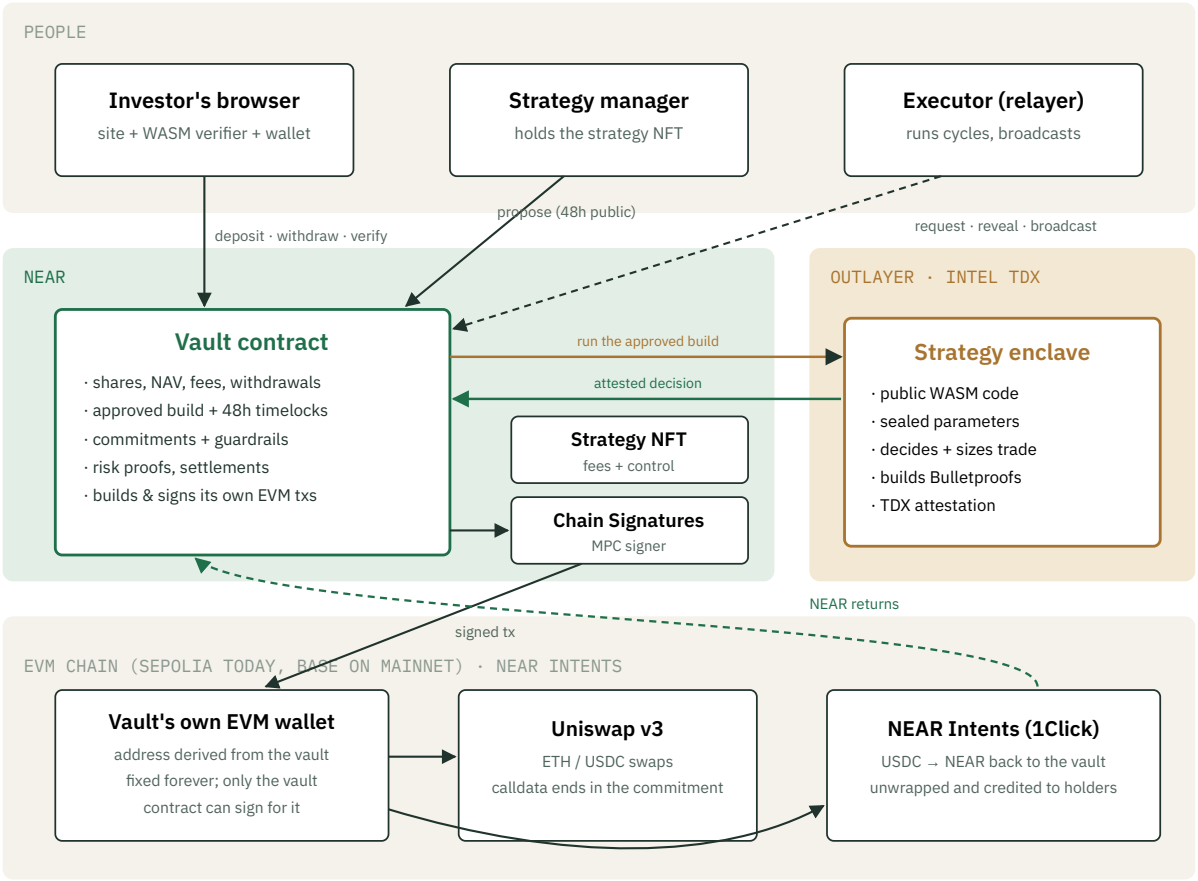


Figure 1. Architecture. The investor's browser talks to the vault contract and verifies everything locally. The enclave decides and proves; the vault enforces and signs; Chain Signatures lets the vault control its own EVM wallet; NEAR Intents brings value home.

Part	Runs on	One-line job
Vault contract	NEAR (Rust, near-sdk)	Holds the money and enforces every rule.
Strategy enclave	OutLayer, Intel TDX (WASM)	Decides privately and proves it behaved.
Chain Signatures	NEAR MPC network	Lets the vault contract sign Ethereum transactions.
Uniswap v3	Sepolia today, Base on mainnet	Where trades execute.
NEAR Intents	1Click API, mainnet	Brings cross-chain value back as NEAR.
Executor	Any machine (Node.js)	Starts cycles and broadcasts what the vault signs.

06 · ONE DECISION, END TO END

A cycle in attested mode

This is the strongest path: no relayer stands between the enclave and the vault. It ran live on 13 November on the rehearsal vault.

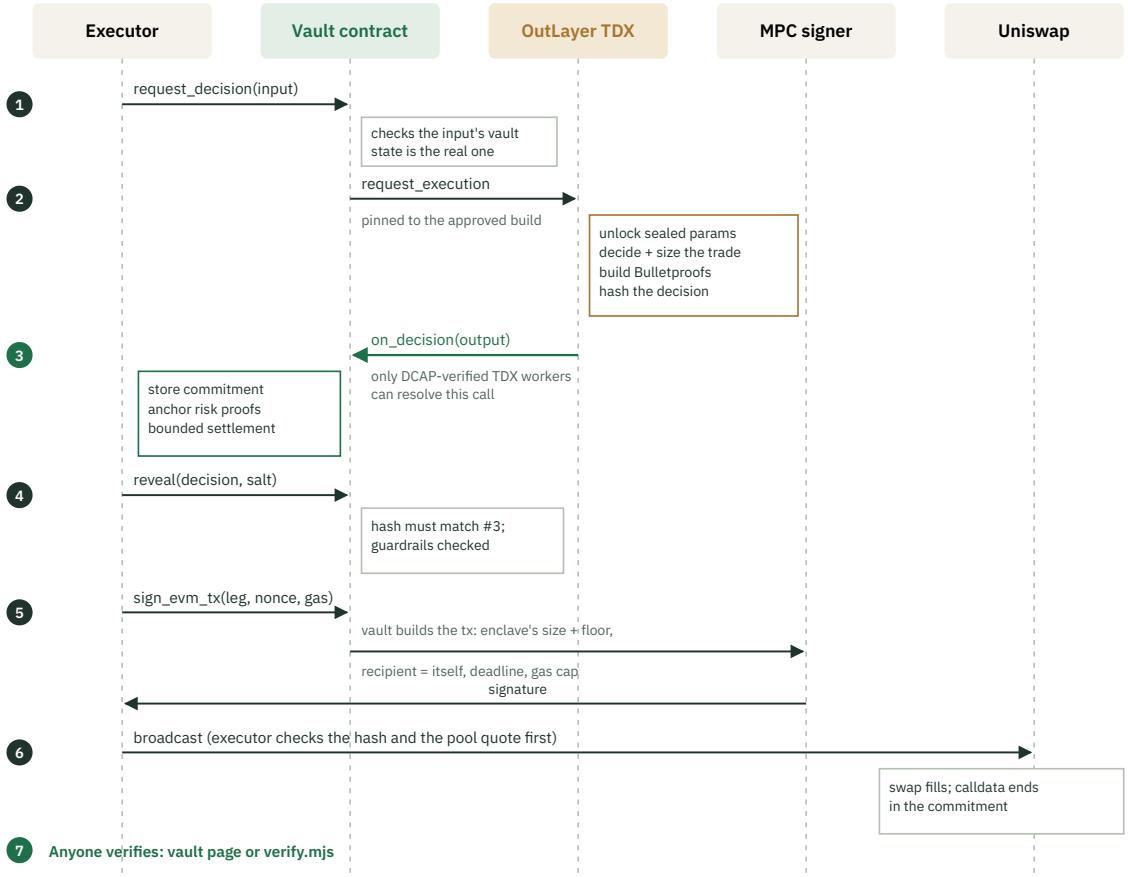


Figure 2. One attested cycle. The enclave's answer goes straight into the vault's callback, so nobody can alter or withhold-and-replace it. The executor can only reveal a decision whose hash is already fixed, and broadcast a transaction the vault built and signed.

Relayed mode, the mode live on the main vault today, differs only in steps 1 to 3: the executor calls OutLayer over HTTPS and relays the output. The relayer then pays for every byte it writes, and the attestation is checked off-chain by anyone. Steps 4 to 7 are identical.

07 · HOW EACH PART WORKS

Components, and what each one solves

For every part: what it is, how it works, and the specific problem it removes.

Strategy: public code, sealed parameters

`strategy/src/lib.rs · compiled to wasm32-wasip1`

A strategy is a small Rust program compiled to WebAssembly. It takes recent prices, the vault's real holdings and its state, and returns a decision. Ours is a momentum rule (fast and slow moving averages) with five secret parameters: the two window lengths, a signal threshold, a position size and a leverage target.

Since strategy v2, the enclave also decides the trade itself: the exact amount in, a slippage floor at the oracle price minus 2%, and the pool fee tier, keeping 0.01 ETH back for gas. It refuses to produce proofs or a valuation when prices are missing or invalid.

SOLVES

Copycats: the logic is readable, the numbers are not. And the relayer can't resize trades, because the size comes from inside the enclave.

OutLayer enclave and attestation

`Intel TDX · OutLayer (testnet: outlayer.testnet · mainnet: outlayer.near)`

OutLayer runs WASM inside Intel TDX machines. The parameters are stored as an encrypted OutLayer secret, locked to the strategy's exact build hash and to a whitelist of callers. The hardware decrypts them only inside a TDX machine running that exact build; change one byte of code and the secret won't open.

Every run produces an attestation naming the executed build, input hash, output hash and a TDX quote. On the on-chain path, only DCAP-verified worker keys can resolve the request, so the delivery itself is attested.

SOLVES

"Which code is running?" The answer is a hardware signature, not a promise. Even the machine's operator can't read the parameters.

Commit, then reveal

`vault: commit_decision · on_decision · reveal`

The enclave outputs `sha256(decision || salt)` first. The vault stores it with the block height before anything trades. The decision is revealed afterwards and accepted only if it hashes to the stored commitment; a swap's calldata ends in the same 32 bytes, tying the on-chain trade to the committed decision.

SOLVES

Back-filled winners and hidden losers. The ledger is complete because nothing can trade without a prior commitment.

Guardrails and Bulletproofs

`vault: check_guardrails · check_input_state · crates/proofs`

Each vault has public guardrails: maximum leverage (3x), maximum drawdown (20%), maximum position size and an allowed-pairs list. The contract checks every revealed decision against them and marks breaches as rejected; a rejected decision can never be signed. It also checks that the enclave's input described the real vault state, so the strategy can't be fed a fake NAV.

For metrics that stay private, the enclave commits to them with Pedersen commitments and builds aggregated Bulletproofs showing each value is at most its limit. The browser verifies them with the same proof library compiled to WebAssembly.

SOLVES

"Max 3x leverage" stops being a sentence and becomes a rule enforced on-chain and proven mathematically, without revealing the actual leverage.

The vault contract

`contracts/vault/src/lib.rs · near-sdk 5 · 413 KB`

The vault issues shares for NEAR deposits and redeems them for an exact slice of its NEAR pool plus a pro-rata claim on cross-chain value. It tracks NAV per share, a high-water mark, and a 20% performance fee minted only on realized gains. It holds the approved build hash, and every change (strategy, mode, code, closing) is public for 48 hours and bound to whoever proposed it.

Deposits open only while everything is held as NEAR, so a valuation report never prices a deposit. If no decision lands for 24 hours, anyone can trigger a one-way emergency exit.

SOLVES

Money safety without trusting the operator: withdrawals, fees and upgrades all follow public rules nobody can bypass once the account keys are deleted.

Chain Signatures: the vault trades for itself

`vault: sign_evm_tx · NEAR MPC (v1.signer)`

NEAR's MPC network lets a contract control an address on another chain. The vault's Ethereum address is derived from its own account and a fixed path; nobody else can sign for it, and it can never be changed. To trade, the contract itself builds the EIP-1559 transaction (WETH, USDC or the Uniswap router only; recipient itself; per-leg gas caps; a deadline within the hour), hashes it, and asks the MPC network to sign. Each leg is signed at most once per decision.

SOLVES

A relayer with its own wallet could redirect funds. Here the relayer only broadcasts transactions the vault built, and checks the hash before it does.

NEAR Intents: the way back

`vault: propose_exit · sign_exit_tx · unwrap_and_repatriate · executor/intents.mjs`

Cross-chain USDC returns as NEAR through NEAR Intents' 1Click API. The manager proposes an exit naming a 1Click deposit address; it is public for 24 hours so anyone can confirm the quote's recipient is the vault. Then the vault signs exactly one USDC transfer. When NEAR Intents delivers wNEAR, anyone can call `unwrap_and_repatriate`, which unwraps it and credits holders.

A live quote priced the vault's 57.94 USDC at 16.48 NEAR, delivered in about 45 seconds.

SOLVES

"How does my money come home?" Value on another chain is no longer stranded, and the exit itself is public before it happens.

Executor

`executor/run.mjs · evm.mjs · intents.mjs · verify.mjs`

A Node.js process that starts each cycle: fetches prices (Pyth, then CoinGecko), asks for a decision, reveals it, asks the vault to sign each leg, checks the vault's transaction hash and the pool's quote, and broadcasts. In attested mode it calls the vault, and the vault calls OutLayer. It pays for every byte it writes to the vault.

SOLVES

Liveness. It is deliberately powerless: it cannot choose amounts, recipients or decisions, and if it stops, holders can exit.

Site, verifier and the strategy NFT

`frontend/ · verify.js · pkg/ (WASM) · NEP-171`

A static site: a registry of vaults, a page per vault with its decision ledger, NAV chart and terms, wallet sign-in for deposits and withdrawals, and a "Counterfeit Bureau" where you can try to cheat (fake a trade, demand 10x leverage, swap the code) and watch each attempt fail. **Run all checks** runs the four seals in the browser; `verify.mjs` runs the same plus two more from a terminal.

The strategy is a NEP-171 NFT. Its holder receives the fees and is the only one who can propose changes; a pending proposal lapses if the NFT is sold.

SOLVES

Verification for non-experts, and a tradable, verifiable track record for strategy creators.

08 · TRUST MODEL

Who can do what

We say exactly what each party is still trusted for. The two modes differ only in how a decision reaches the vault.

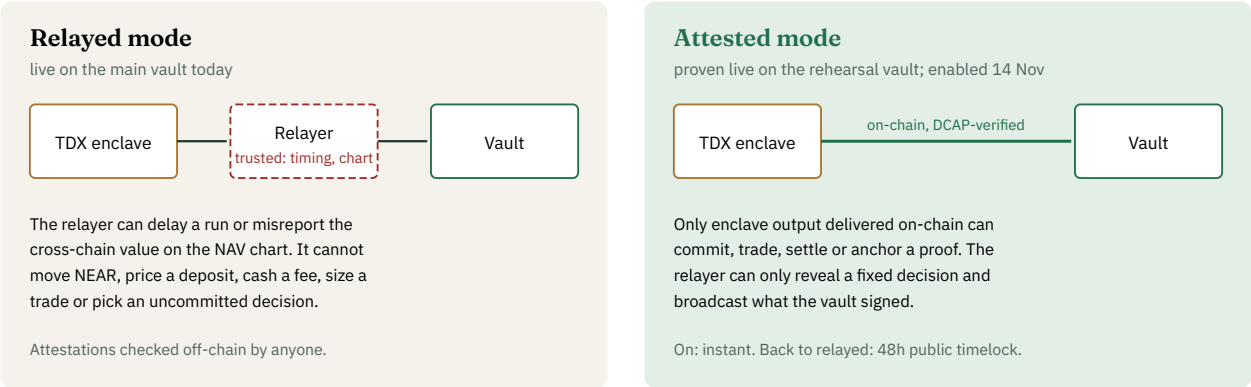


Figure 3. Trust boundaries. In attested mode the relayer leaves the decision path entirely.

Party	Can	Cannot
Strategy manager (NFT holder)	Propose a new strategy, mode change, code upgrade, exit or close. Each is public 48h (exit: 24h) before it applies.	Change anything instantly, withdraw other people's money, or keep a proposal alive after selling the NFT.
Executor (relayer)	Start cycles, reveal committed decisions, choose EVM nonce and gas price (capped), broadcast.	Choose trade size, slippage floor or recipient; sign twice; price deposits; cash fees; write for free.
OutLayer / TDX	Run the approved build and attest to it.	Run a different build with the same secrets; let the operator read parameters.
Investor	Deposit (while fully in NEAR), withdraw any time, trigger emergency exit after 24h of silence, verify everything.	Be diluted by a deposit priced on a valuation report.
Anyone	Verify, trigger emergency exit, apply a matured proposal, unwrap returned NEAR, close an empty vault after notice.	Anything that moves money to themselves.

Rules that protect the money

Guarantee	How it's enforced and tested
A withdrawal pays an exact pro-rata slice of the NEAR pool and never reads a valuation	`withdraw`; fuzz invariant S5
The contract can always pay what it owes	fuzz S2 (solvency against a model of NEAR held); live check 6
A valuation can never price a deposit	deposits only while everything is in NEAR; fuzz S8
Fees only on realized gains above the high-water mark	`crystallize_fees`; fuzz S7
Paid-out cross-chain value never exceeds what came home	fuzz S3; units conserved (S4)
The vault's EVM wallet is fixed; swaps pay only the vault	no rotation exists; recipient hard-wired; calldata test byte-identical in Rust and JavaScript
The locked storage deposit only returns from an empty vault	`propose_close` (48h) then `close_vault`; contract deletes its own account
Nobody can point a vault at another chain	network chosen from the vault's account name; no setter exists

Two independent reviews

Review 1 (v3) found three critical issues: the executor could drain the pool through misreported valuations; the cross-chain wallet belonged to the executor, not the vault; claims could overpay. v4 fixed all three.

Review 2 (v4) asked what the relayer could still do. It found that valuations still affected deposit pricing and fees, the relayer chose trade sizes, the wallet could be rotated, and the account's full-access key could replace the code. v5 answers each: deposits and fees no longer read valuations, the enclave sizes trades, rotation is gone, upgrades go through a public 48-hour path, and attested mode removes the relayer from the decision path. Two earlier documentation claims that the review showed to be wrong were corrected publicly.

> **Found while deploying.** The v4 code was 853 KB with debug names left in, which locked more storage stake than the vault's spare balance, leaving it briefly unable to cover its pool. The stripped build is 413 KB; the vault is solvent and check 6 now guards it.

Testing. 56 Rust tests: unit tests for every rule above, a full pipeline test (strategy to contract), and a property-based fuzz that runs hundreds of random deposit, withdrawal, settlement, repatriation and fee sequences against eight invariants. Four executor tests include a full cycle against a local enclave.

10 · LIVE EVIDENCE

What has actually happened on-chain

Every row is a public transaction anyone can look up.

Event	Where
Main vault (relayed mode)	`vault.alpha-vaults-23.testnet`, 19 committed decisions; EVM wallet `0x63fbAd3b…9B6f`
Rehearsal vault (attested)	`vault2.alpha-vaults-23.testnet`; EVM wallet `0x594441E8…8696`
Decision delivered on-chain by TDX to the vault's callback	NEAR tx `CpUKMrgDPGwCWXEmmQFhyfv4sLb22c6WY6bttZDm5527` (commitment #18)
Fully attested cycle with a trade	request `6CD9wME5tHnHRRhwZkvZQ3j62kTgTV7TvHXjEEJzTQro`; Sepolia swap `0xa7ca6d0e…13a1`: sold 0.01308 ETH, size and 38.46 USDC floor chosen by the enclave; calldata ends in commitment `55b77309…`
v5 contract deployed and migrated in one atomic transaction	`EGUedioDCU5v8GJptkCN5HaKZFeCRG6DmiNkvGndFTWJ`
Strategy v2 proposed (public 48h)	`HE8s5eQL…`, applicable 14 Nov 17:51 UTC
Returned NEAR unwrapped and credited to holders	`AFfuBsgvysevy8JVU5YRRB73ie5y14G5PubfqZ99kZUK` (0.04875 NEAR)
NEAR Intents live quote	57.94 USDC → 16.48 NEAR, about 45 s (1Click dry run)

Check it yourself

- **Browser:** open a vault page and press **Run all checks**.
- **Terminal:** `cd executor && npm ci && node verify.mjs`. Six checks, read-only, no keys; exit code 0 only if all pass.
- **Document:** `VERIFY.md` lists every claim with the code that enforces it, the test that attacks it and a live transaction.

What we don't claim

- **Testnet only, so far.** Sepolia's pools are mispriced about 10× against real ETH, so only sells fill. The mainnet build trades on Base and is ready to deploy as a manager-only vault.
- **The requester supplies market prices,** in both modes. The contract checks the vault-state part of the input but not prices; the price check compares them with public history.
- **The NEAR Intents send half is mainnet-only.** NEAR Intents has no testnet. The arrival half ran live on testnet.
- **Decisions become public when executed.** They have to, so they can be checked. Someone could study the history and try to infer the parameters.
- **No professional audit yet.** Two independent AI-assisted reviews; every finding and remaining risk is in `docs/SECURITY.md`.
- **One example strategy** (momentum), and no self-serve way yet for designers to launch a vault.

Roadmap

Next	What it adds
Mainnet vault on Base	Real money, a real NEAR Intents round trip, a manager-only cap at launch.
Enclave-fetched prices	The enclave fetches its own market data over HTTPS, removing the last input the requester controls.
Vault factory and creator UI	Strategy designers upload a build, seal parameters and launch a vault from a web form.
Strategy marketplace	Buy and sell strategy NFTs with their verified records attached.
Professional audit	Before opening vaults to outside deposits.

12 · GLOSSARY

Terms

Attestation	A hardware-signed statement from an Intel TDX machine naming the exact code it ran and the hashes of its input and output.
Bulletproofs	Short zero-knowledge proofs that a hidden number lies in a range, used here to show leverage and drawdown stayed under their limits.
Chain Signatures	NEAR's MPC network that lets a NEAR contract sign transactions for addresses on other chains.
Commitment	A hash of a decision plus a random salt, recorded before the decision is revealed.
Guardrails	A vault's public risk limits, enforced by the contract.
High-water mark	The highest NAV per share reached; fees are only charged above it.
NAV	Net asset value: what the vault holds, in NEAR; NAV per share sets deposit and withdrawal prices.
NEAR Intents (1Click)	NEAR's cross-chain swap network; 1Click gives a deposit address that swaps funds and delivers them to a recipient.
OutLayer	A NEAR network that runs WebAssembly inside Intel TDX secure machines, callable on-chain or over HTTPS.
Relayed / attested mode	Whether enclave output reaches the vault through a named relayer, or directly through OutLayer's on-chain callback.
Strategy NFT	A NEP-171 token whose holder earns the vault's fees and proposes changes.
TEE / Intel TDX	A trusted execution environment: hardware that runs code so even the machine's operator can't see or alter it.
Timelock	A public waiting period before a change can apply, so anyone who disagrees can leave first.

Verifiable Alpha Vaults · built for the NEAR Hackathon 2026 · source: github.com/amudhan23/verifiable-alpha-vaults · this document describes the system as of 13 November 2026.